

Preprocess and Encode Data

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report, precision_score

# 1. Load the dataset
df = pd.read_csv("/content/Invistico_Airline.csv.csv")
df = df.dropna() # Safely handle missing survey answers
df.head()
```

	satisfaction	Customer Type	Age	Type of Travel	Class	Flight Distance	Seat comfort	Departure/Arrival time convenient
0	satisfied	Loyal Customer	65	Personal Travel	Eco	265	0	
1	satisfied	Loyal Customer	47	Personal Travel	Business	2464	0	
2	satisfied	Loyal Customer	15	Personal Travel	Eco	2138	0	
3	satisfied	Loyal Customer	60	Personal Travel	Eco	623	0	
4	satisfied	Loyal Customer	70	Personal Travel	Eco	354	0	

5 rows × 22 columns

```
# 2. Convert target variable 'satisfaction' to binary (1 for Satisfied, 0 for Dissatisfied)
df['satisfaction'] = df['satisfaction'].map({'satisfied': 1, 'dissatisfied': 0})
# Handle case sensitivities if any
df['satisfaction'] = df['satisfaction'].fillna(0).astype(int)
```

```
# 3. Encode categorical features into numerical 0 and 1 values
X = df.drop(columns=['satisfaction'])
y = df['satisfaction']
X_encoded = pd.get_dummies(X, drop_first=True)
```

```
# 4. Split data into Training (80%) and Testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X_encoded, y, test_size=0.2, random_state=42)

print("Data successfully prepared and split!")
print(f"Training features shape: {X_train.shape}")
```

Data successfully prepared and split!
 Training features shape: (103589, 22)

Fit Model and Generate Evaluation Metrics

1. Initialize and train the Logistic Regression Model

```
log_reg = LogisticRegression(max_iter=1000)
```

```
log_reg.fit(X_train, y_train)
```

/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: Converge
 STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

```
    LogisticRegression
    LogisticRegression(max_iter=1000)
```

2. Predict on the test set

```
y_pred = log_reg.predict(X_test)
```

```
y_pred
```

```
array([1, 0, 0, ..., 0, 1, 0])
```

3. Generate Evaluation Metrics

```
cm = confusion_matrix(y_test, y_pred)
```

```
precision = precision_score(y_test, y_pred)
```

```
recall = recall_score(y_test, y_pred)
```

```
print("--- MODEL EVALUATION METRICS ---")
```

```
print("Confusion Matrix:")
```

```
print(cm)
```

```
print(f"\nPrecision Score: {precision:.4f}")
```

```
print(f"\nRecall Score: {recall:.4f}")
```

```
print("\nDetailed Classification Report:")
```

```
print(classification_report(y_test, y_pred))
```

```
--- MODEL EVALUATION METRICS ---
```

```
Confusion Matrix:
```

```
[[ 9240 2581]
 [ 2271 11806]]
```

```
Precision Score: 0.8206
```

```
Recall Score: 0.8387
```

```
Detailed Classification Report:
```

	precision	recall	f1-score	support
0	0.80	0.78	0.79	11821

	1	0.82	0.84	0.83	14077
accuracy				0.81	25898
macro avg		0.81	0.81	0.81	25898
weighted avg		0.81	0.81	0.81	25898

✓ Interpret Model Coefficients

To understand which factors contribute most to passenger satisfaction, we'll examine the coefficients of our trained Logistic Regression model. In a logistic regression model, the coefficients represent the change in the *log-odds* of the dependent variable (satisfaction in our case) for a one-unit change in the independent variable, holding all other variables constant.

- **Positive coefficients** indicate that as the feature value increases, the log-odds of being 'satisfied' increase (i.e., higher likelihood of satisfaction).
- **Negative coefficients** indicate that as the feature value increases, the log-odds of being 'satisfied' decrease (i.e., lower likelihood of satisfaction).

```
# Extract coefficients and feature names
coefficients = log_reg.coef_[0]
feature_names = X_encoded.columns

# Create a DataFrame for better readability
coef_df = pd.DataFrame({'Feature': feature_names, 'Coefficient': coefficients})

# Sort by the absolute value of the coefficient to see most influential features
coef_df['Abs_Coefficient'] = abs(coef_df['Coefficient'])
coef_df = coef_df.sort_values(by='Abs_Coefficient', ascending=False)

print("Top 10 most influential features on passenger satisfaction:")
display(coef_df.head(10))
```

Top 10 most influential features on passenger satisfaction:

	Feature	Coefficient	Abs_Coefficient	
18	Customer Type_disloyal Customer	-2.107339	2.107339	
20	Class_Eco	-1.276214	1.276214	
21	Class_Eco Plus	-0.859741	0.859741	
7	Inflight entertainment	0.578401	0.578401	
19	Type of Travel_Personal Travel	-0.566749	0.566749	
9	Ease of Online booking	0.460416	0.460416	
4	Food and drink	-0.276738	0.276738	
10	On-board service	0.269084	0.269084	
2	Seat comfort	0.256008	0.256008	
13	Checkin service	0.187989	0.187989	